

Unity Catalog & Delta Lake 101

Databricks Free Edition — PySpark reference

Two systems work together whenever you run PySpark on Databricks, and it's easy to blur them together:

- Delta Lake — a storage format. It decides how data sits on disk (Parquet files + a transaction log).
- Unity Catalog (UC) — a governance layer. It decides how that data is named, discovered, and permissioned.

A UC table is basically "a name UC lets you use" pointing at "a folder of Delta files." Understanding both layers separately is what makes storage-location errors make sense — and answers the two questions you asked: how reads/writes work on each layer, and whether you must write data before registering it as a table.

Part 1: Delta Lake architecture

A Delta table is just a directory containing two things:

```
/some/path/sales_raw/  
├── _delta_log/           ← the transaction log (source of truth)  
│   ├── 00000000000000000000.json  
│   ├── 00000000000000000001.json  
│   └── ...  
├── part-0001-xxxx.snappy.parquet ← actual data, plain Parquet  
├── part-0002-xxxx.snappy.parquet  
└── ...
```

- Parquet files hold the actual rows/columns. Nothing special — any Parquet reader can open them.
- `_delta_log/` holds an ordered log of JSON commits. Each commit says which Parquet files are "active" as of that version.

That log is what gives Delta its features: ACID transactions (a write either fully commits or doesn't happen), time travel (query an earlier version, since old files aren't deleted until you VACUUM), and schema enforcement/evolution (the log tracks schema per version).

Delta Lake doesn't know or care about Unity Catalog — it works purely by path.

Reading and writing Delta Lake directly (path-based)

This is "unmanaged" Delta — no catalog involved at all, just a folder path.

```
# WRITE — creates the folder structure above at this exact path  
raw_df.write.format('delta') \  
    .mode('overwrite') \  
    .option('overwriteSchema', 'true') \  
    .save('/Volumes/workspace/bronze/data/sales_raw')  
  
# READ — just needs the path  
df = spark.read.format('delta').load('/Volumes/workspace/bronze/data/sales_raw')  
  
# APPEND new rows instead of overwriting  
new_rows_df.write.format('delta').mode('append').save('/Volumes/workspace/bronze/data/sales_raw')  
  
# TIME TRAVEL — read an older version of the same path  
old_df = spark.read.format('delta') \  
    .load('/Volumes/workspace/bronze/data/sales_raw') \  
    .versionAsOf(1)
```

```

.option('versionAsOf', 3) \
.load('/Volumes/workspace/bronze/data/sales_raw')

# MERGE (upsert) using the Delta table API, still path-based
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, '/Volumes/workspace/bronze/data/sales_raw')
dt.alias('t').merge(
    new_rows_df.alias('s'), 't.order_id = s.order_id'
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

```

Every one of these calls works with zero mention of a catalog, schema, or table name — Delta only cares about the path.

Part 2: Unity Catalog architecture

UC adds a naming and permission hierarchy on top of storage. The object tree, top to bottom:

```

Metastore (one per Databricks account/region – you don't create this in Free Edition)
├── Catalog      e.g. "workspace" (your default catalog in Free Edition)
│   └── Schema   e.g. "bronze" (equivalent to a "database")
│       ├── Table e.g. "sales_raw" (rows/columns – usually Delta)
│       ├── View  e.g. "sales_valid" (a saved query, no storage of its own)
│       ├── Volume e.g. "raw_files" (non-tabular files: CSVs, images, checkpoints)
│       └── Function

```

Every data object is referenced with a three-level name: catalog.schema.table.

Free Edition specifics (this matters for you)

- You're on a serverless-only workspace — no cluster to configure.
- A catalog called "workspace" is pre-created, backed by Databricks-managed "default storage." You never touch a cloud bucket, ARN, or storage credential.
- Because there's no external storage config available, you cannot create true "external tables" the way you could on a paid workspace with an External Location registered — this is exactly why `LOCATION 'abfss://...' / 's3://...'` isn't an option, and why `LOCATION` pointing at a Volume path throws an error.
- Volumes and managed Tables both use default storage, but UC will never let a table's location sit inside a Volume's path, or vice versa.

Practical conclusion for Free Edition: use managed tables (`saveAsTable`) for anything you want governed, and use Volumes only for genuinely non-tabular files — not as a substitute table location.

Reading and writing Unity Catalog tables

These calls go through the catalog's name resolver first, which finds where the Delta files live, then hand off to the Delta engine underneath.

```

spark.sql("USE CATALOG workspace")

# WRITE – creates + registers a managed table in one call
raw_df.write.mode('overwrite').format('delta') \
    .option('overwriteSchema', 'true') \
    .saveAsTable('workspace.bronze.sales_raw')

# APPEND to an existing catalog table
new_batch_df.write.mode('append').saveAsTable('workspace.bronze.sales_raw')
# ...or equivalently:

```

```

new_batch_df.write.insertInto('workspace.bronze.sales_raw')

# READ – by catalog name, two equivalent styles
df = spark.table('workspace.bronze.sales_raw')
df = spark.read.table('workspace.bronze.sales_raw')

# READ – plain SQL
spark.sql("SELECT * FROM workspace.bronze.sales_raw LIMIT 10").show()

# TIME TRAVEL still works on catalog tables
old_df = spark.read.option('versionAsOf', 3).table('workspace.bronze.sales_raw')

# DDL – create/alter/drop go through spark.sql
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.silver")
spark.sql("DROP TABLE IF EXISTS workspace.bronze.sales_raw")

# Inspect what UC knows about, without SQL
spark.catalog.listCatalogs()
spark.catalog.listTables("bronze")

```

Part 3: Instantiating each component

All of this can be done in SQL (`spark.sql(...)`) or with catalog-aware PySpark calls — they're interchangeable.

Catalog

```

spark.sql("CREATE CATALOG IF NOT EXISTS workspace") # usually already exists in Free Edition
spark.sql("USE CATALOG workspace")

```

Schema (database)

```

spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.bronze")

```

Volume (for raw, non-tabular files only)

```

spark.sql("CREATE VOLUME IF NOT EXISTS workspace.bronze.raw_files")
# Usable as a normal path afterwards:
# /Volumes/workspace/bronze/raw_files/incoming.csv

```

Managed table

```

raw_df.write.mode('overwrite').format('delta') \
    .option('overwriteSchema', 'true') \
    .saveAsTable('workspace.bronze.sales_raw')

```

View — no storage, just a saved query

```

spark.sql("""
CREATE OR REPLACE VIEW workspace.bronze.sales_valid AS
SELECT * FROM workspace.bronze.sales_raw WHERE amount IS NOT NULL
""")

```

Part 4: Do you need to `save()` first, then `saveAsTable()`?

Short answer: not for normal use. `saveAsTable()` both writes the Delta files AND registers the catalog name in a single call — one physical copy, one step. UC picks the storage path for you (inside its managed default storage), so there's nothing to save beforehand.

You only need two separate writes when you genuinely want the raw files sitting at one specific, exact path (e.g. a Volume, for another tool or process to read directly) while also having a governed table in the catalog. In Free Edition, those two things can never be the same physical location — a table's storage can't overlap a Volume's path, and true external tables (LOCATION pointing anywhere you choose) aren't available. So if you need both, you end up with two physical copies of the data:

1. Write the raw files to the exact path you need (a Volume), by path — this is a plain Delta/CSV/Parquet write, no catalog involved.
2. Separately, read that data back and call `saveAsTable(...)` to register a second, UC-managed copy under `catalog.schema.table`.

```
# Step 1 – raw landing zone, exact path you control, no catalog
raw_df.write.format('delta').mode('overwrite') \
    .save('/Volumes/workspace/bronze/raw_files/sales_raw')

# Step 2 – read it back, register a governed copy in the catalog
landed_df =
spark.read.format('delta').load('/Volumes/workspace/bronze/raw_files/sales_raw')

landed_df.write.mode('overwrite').format('delta') \
    .option('overwriteSchema', 'true') \
    .saveAsTable('workspace.bronze.sales_raw')
```

This does mean duplicated storage — two copies of the same data — so it's only worth doing when something outside your table genuinely needs file-level access to the raw path. If nothing does, skip step 1 entirely and just `saveAsTable()` straight from the source DataFrame; that's the common case and the one used throughout this guide.

Part 5: How PySpark talks to each layer

You want to...	Call	Talks to
Run DDL (catalog/schema/table/view)	<code>spark.sql("CREATE ...")</code>	Unity Catalog
Write a DataFrame as a governed table	<code>df.write.saveAsTable("cat.schema.tbl")</code>	UC (registers) + Delta (stores)
Read a governed table	<code>spark.table(...)</code> / <code>spark.read.table(...)</code>	UC (resolves name) → Delta
Write/read Delta by raw path	<code>df.write.save(path)</code> / <code>spark.read.load(path)</code>	Delta only, UC bypassed
List/inspect catalog objects	<code>spark.catalog.listTables(...)</code>	UC
Time travel	<code>option("versionAsOf", n)</code>	Delta log, via UC or path

Part 6: End-to-end bronze → silver example

```
spark.sql("USE CATALOG workspace")
spark.sql("CREATE SCHEMA IF NOT EXISTS bronze")
spark.sql("CREATE SCHEMA IF NOT EXISTS silver")

# Bronze: raw ingest, managed table, one call – no separate save() needed
raw_df.write.mode('overwrite').format('delta') \
    .option('overwriteSchema', 'true') \
    .saveAsTable('workspace.bronze.sales_raw')
```

```
# Silver: transform, read via catalog name, write via catalog name
bronze_df = spark.table('workspace.bronze.sales_raw')

silver_df = bronze_df.filter("amount IS NOT NULL").dropDuplicates(["order_id"])

silver_df.write.mode('overwrite').format('delta') \
    .saveAsTable('workspace.silver.sales_clean')

# Query it back with plain SQL, from anywhere in the workspace
spark.sql("SELECT * FROM workspace.silver.sales_clean LIMIT 10").show()
```

No paths, no volumes, no LOCATION clauses anywhere — everything is addressed by `catalog.schema.table`, which is the pattern Free Edition wants you to use.

Quick reference

- Delta Lake = file format (Parquet + `_delta_log`), works with or without a catalog.
- Unity Catalog = naming + governance (`catalog.schema.table`), sits on top of Delta (or other formats).
- Managed table = UC owns the storage location; created via `saveAsTable` or `CREATE TABLE` with no `LOCATION`. Use this in Free Edition.
- External table = you own the storage location via `LOCATION`; requires an External Location credential — not available in Free Edition.
- Volume = UC object for non-tabular files; cannot overlap with a table's storage path.
- View = saved SQL query, no storage of its own.
- Need the same data at an exact path AND in the catalog? That's two physical writes — path-based `save()`, then `saveAsTable()` on the read-back.